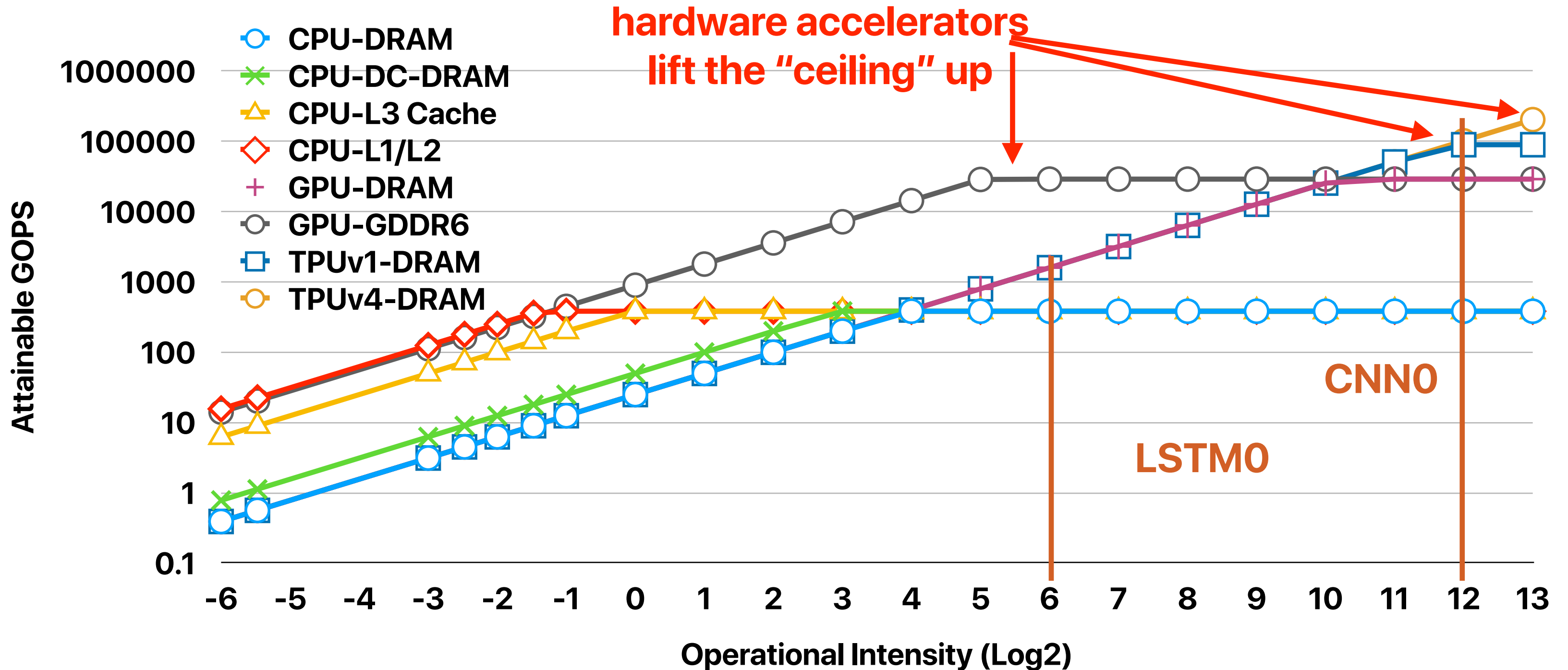


Innovative use of hardware accelerators

Hung-Wei Tseng

Recap: the rooflines of modern systems



Recap: 7 generations of TPUs

	TPU v1	TPU v2	TPU v3	TPU v4	TPU v5 e/p	TPU v6e	TPU v7
Announced	2015	2017	2018	2020	2023	2024	2025
Target	Inference	Training + Inference	Training	Training + Inference	Inference(v5e); Training(v5p)	Training + Inference	Inferencing (LLM)
Peak Performance Per Chip	92 TOPS	45 TFLOPS	420 TFLOPS	1000 TFLOPS	393 TOPS (Int8); 459 TFLOPS (v5p)	~2000 TOPS	4614 TFLOPS
On-chip Memory	28 MB	8 GB	32 GB/chip; 128 GB/board	≥128 GB per board	95 GB	2× v5 HBM capacity	192 GB per chip
Memory bandwidth (GB/s)	167	600	900	1200	2765	1638	7380
Interconnect	PCIe	4 chips per board; 64 per pod	1024+ per pod	4096 per pod with Optical Circuit Switches (OCS)	256-chip pod	Jupiter fabric: 100K chips/pod	~9,216 chips/pod
Power (W)	40	200-280	123-262	125-250	200-300	300	600
Features	256×256 CPU-style systolic array	Introduced bfloat16 training, liquid cooling	Further liquid cooling, higher memory.	Architecture built around 5-D torus + re-configurable optical fabric; Adds SparseCores: ~5% of die foembedding	v5e (inference-optimized, eight "shapes" supporting up to 2T parameter models; v5p (training-oriented)	Trained Google Gemini 2.0; Major leap: 4.7× peak perf per chip over v5, 2× memory and interface bandwidth, 67% energy efficiency gain	First TPU "inference-first" design

Turing Architecture

NVIDIA, "NVIDIA Turing GPU architecture: Graphics reinvented," 2018. [Online]. Available: <https://www.nvidia.com/content/dam/en-zz/Solutions/designvisualization/technologies/turing-architecture/NVIDIATuring-Architecture-Whitepaper.pdf>

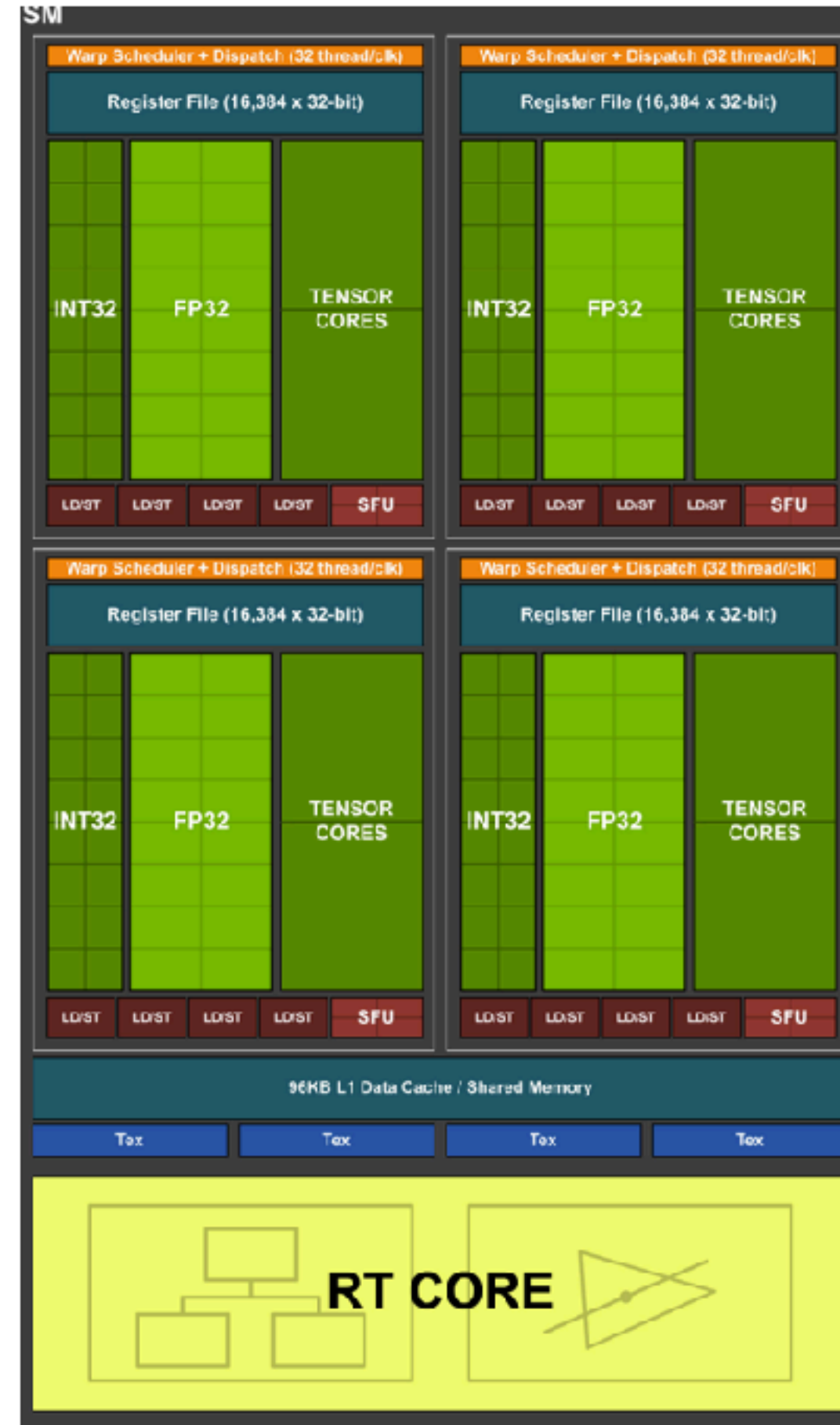
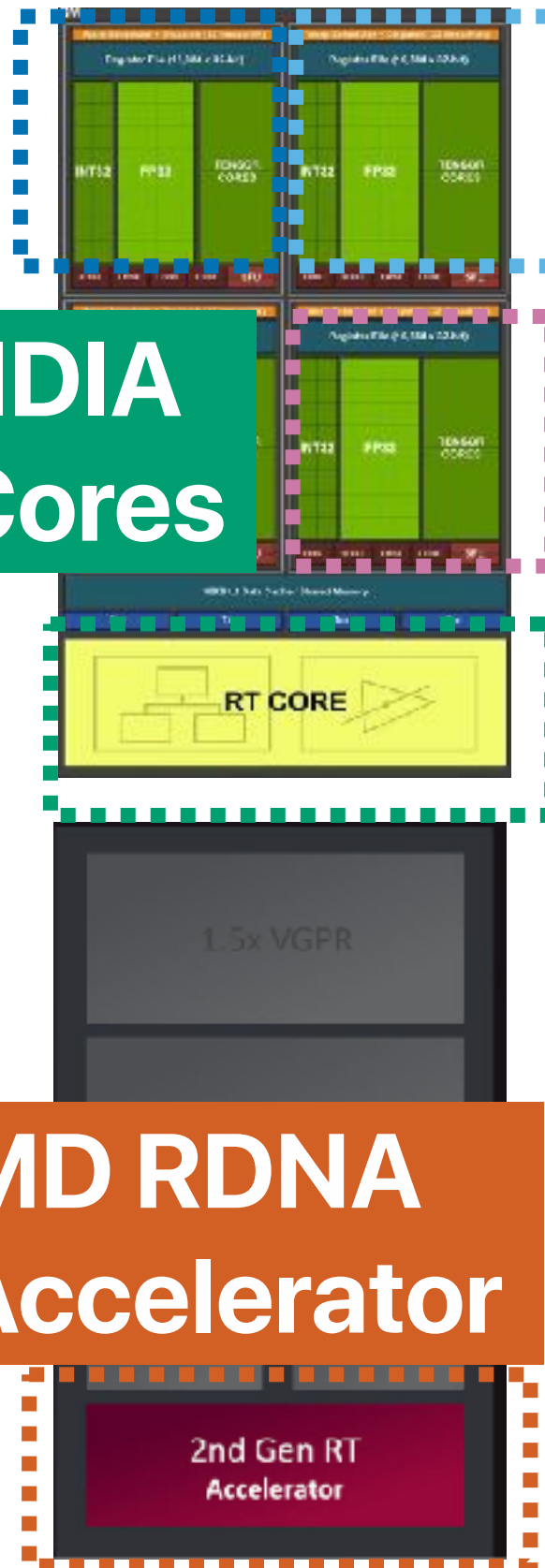


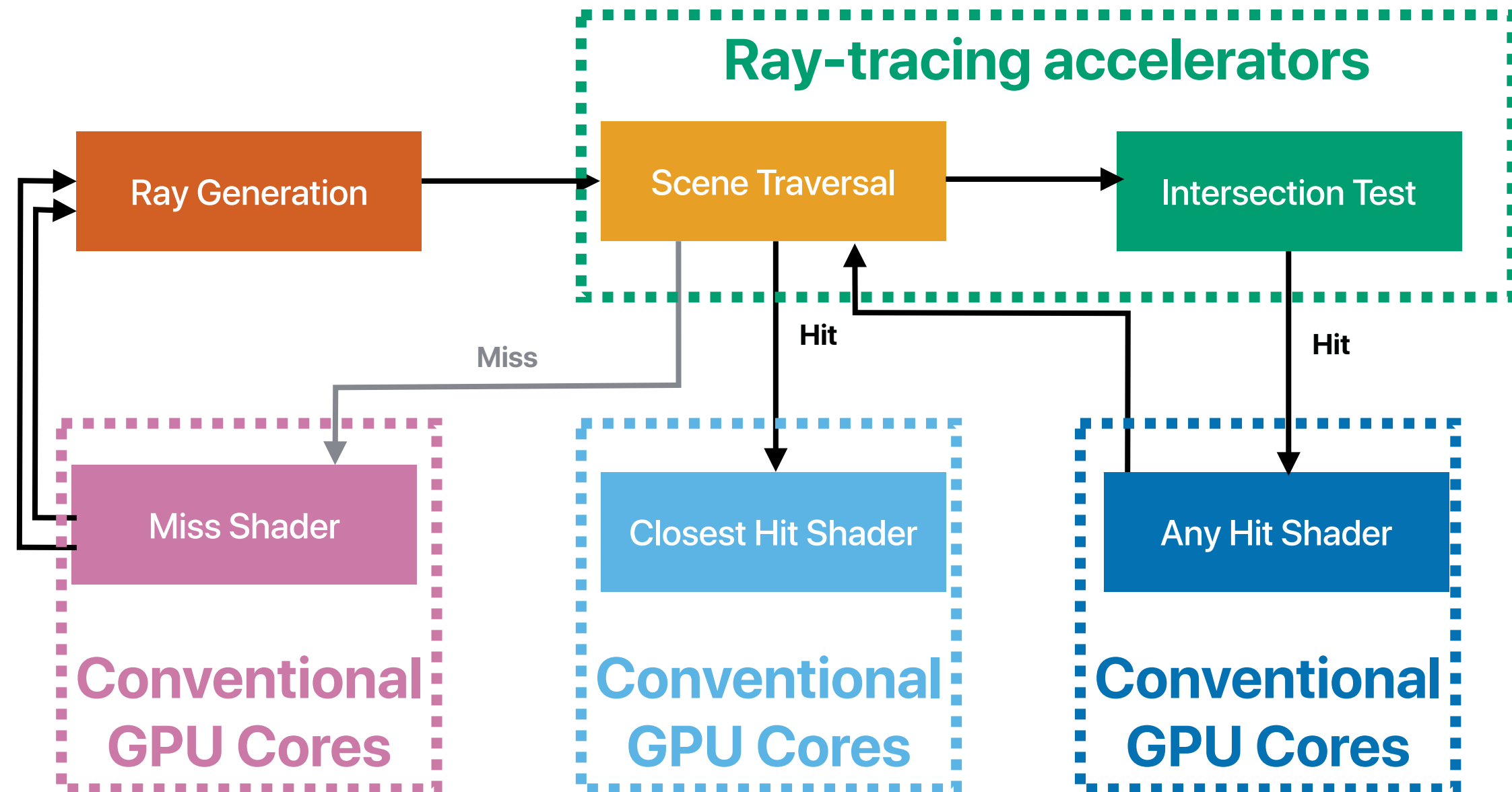
Figure 4. Turing TU102/TU104/TU106 Streaming Multiprocessor (SM)

Ray Tracing Accelerators

**NVIDIA
RT Cores**



**AMD RDNA
RT Accelerator**



Recent development of NVIDIA GPUs

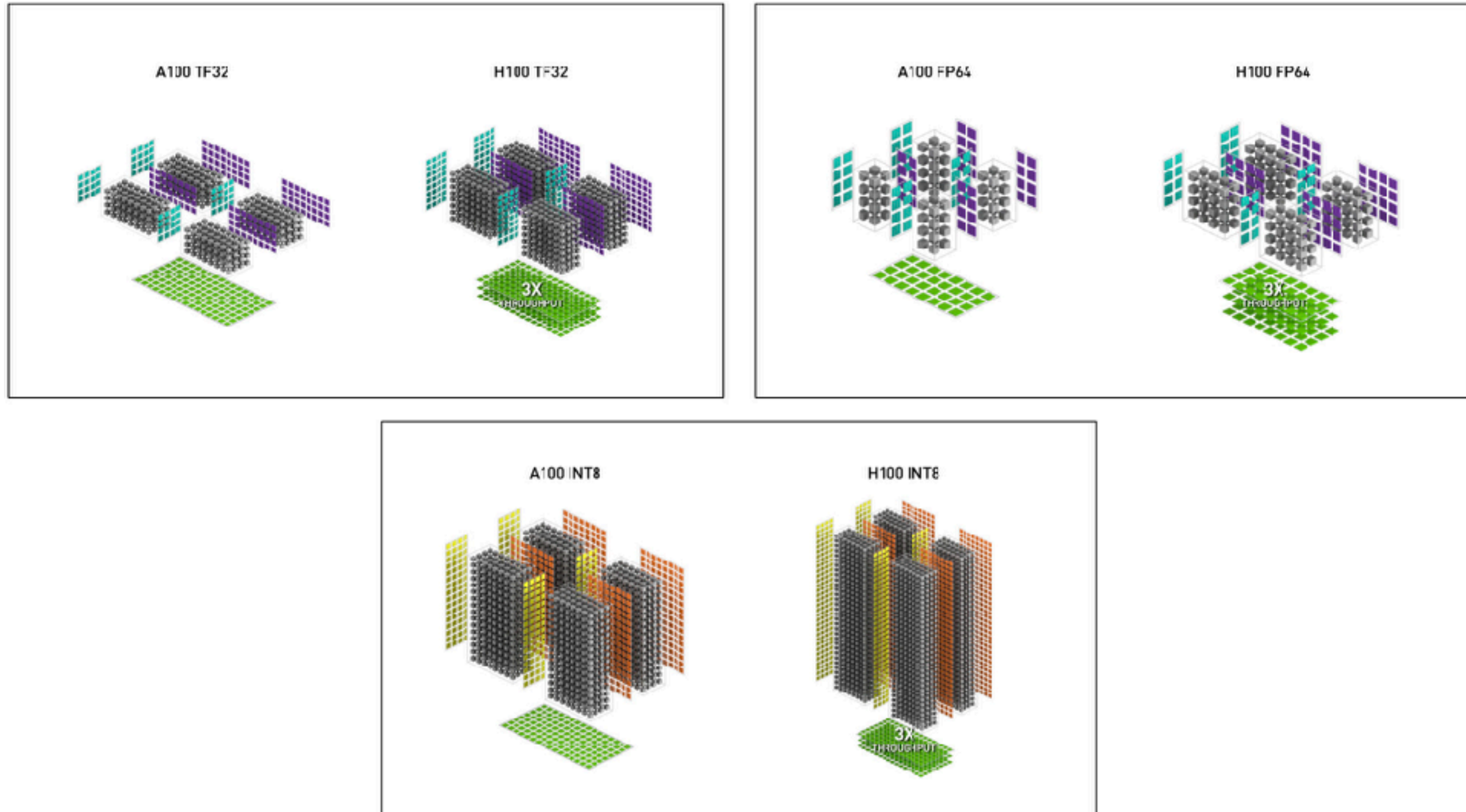


Table 2. H100 speedup over A100 (H100 Performance, TC=Tensor Core)

	A100	A100 Sparse	H100 SXM5	H100 SXM5 Sparse	H100 SXM5 Speedup vs A100
FP8 Tensor Core	NA	NA	1978.9 TFLOPS	3957.8 TFLOPS	6.3x vs A100 FP16 TC
FP16	78 TFLOPS	NA	133.8 TFLOPS	NA	1.7x
FP16 Tensor Core	312 TFLOPS	624 TFLOPS	989.4 TFLOPS	1978.9 TFLOPS	3.2x
BF16 Tensor Core	312 TFLOPS	624 TFLOPS	989.4 TFLOPS	1978.9 TFLOPS	3.2x
FP32	19.5 TFLOPS	NA	66.9 TFLOPS	NA	3.4x
TF32 Tensor Core	156 TFLOPS	312 TFLOPS	494.7 TFLOPS	989.4 TFLOPS	3.2x
FP64	9.7 TFLOPS	NA	33.5 TFLOPS	NA	3.5x
FP64 Tensor Core	19.5 TFLOPS	NA	66.9 TFLOPS	NA	3.4x
INT8 Tensor Core	624 TOPS	1248 TOPS	1978.9 TFLOPS	3957.8 TFLOPS	3.2x

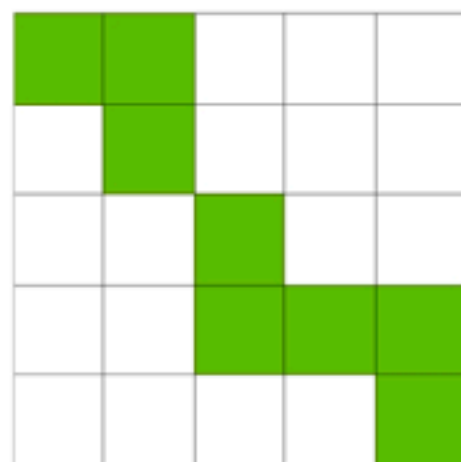
New DPX Instructions for Accelerated Dynamic Programming

Many “brute force” optimization algorithms have the property that a sub-problem solution is reused many times when solving the larger problem. Dynamic Programming is an algorithmic technique for solving a complex recursive problem by breaking it down into simpler sub-problems. By storing the results of sub-problems, without the need to recompute them when needed later, Dynamic Programming algorithms reduce the computational complexity of exponential problem sets to a linear scale.

Dynamic programming is commonly used in a broad range of optimization, data processing, and genomics algorithms. In the rapidly growing field of genome sequencing, the Smith-Waterman dynamic programming algorithm is one of the most important methods in use. In the robotics space, Floyd-Warshall is a key algorithm used to find optimal routes for a fleet of robots through a dynamic warehouse environment in real-time.

H100 introduces DPX instructions to accelerate the performance of Dynamic Programming algorithms by up to 7x compared to Ampere GPUs. These new instructions provide support for advanced fused operands for the inner loop of many DP algorithms. This will lead to dramatically faster times-to-solutions in disease diagnosis, logistics routing optimizations, and even graph analytics.

DYNAMIC PROGRAMMING
Exponential to polynomial time problem solving



A BROAD RANGE OF USE CASES
from genomics to routing optimization



Optimization



Genomics

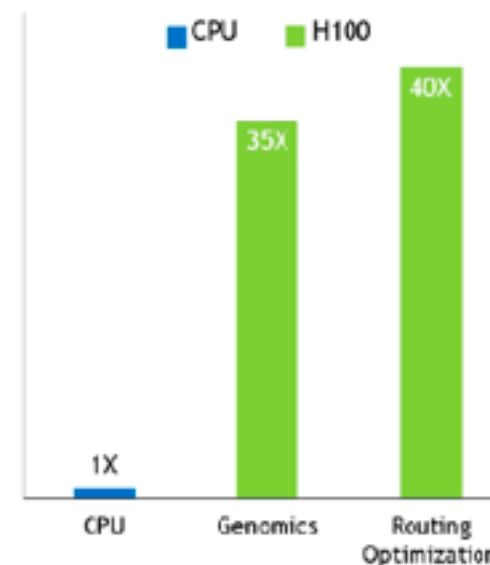


Graph Analytics



Data Processing

REAL-TIME PERFORMANCE
40X speedup



Each chip is now also a collection of accelerators



Introducing Intel® Core™ Ultra Processors for the Edge

https://cdrdv2-public.intel.com/817736/817736_Heterogeneous%20AI%20Powerhouse-Intel%20Core%20Ultra-WP-Rev1.0.pdf

3.0 Introducing Intel® Core™ Ultra Processors for the

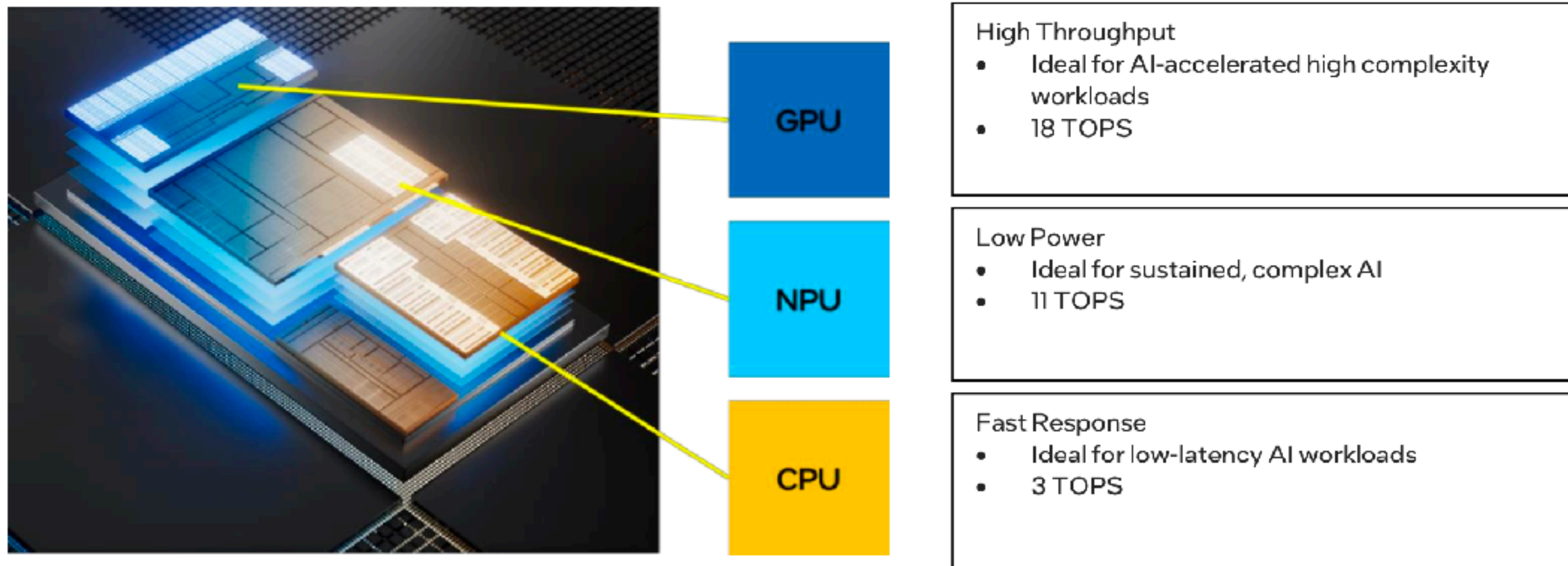
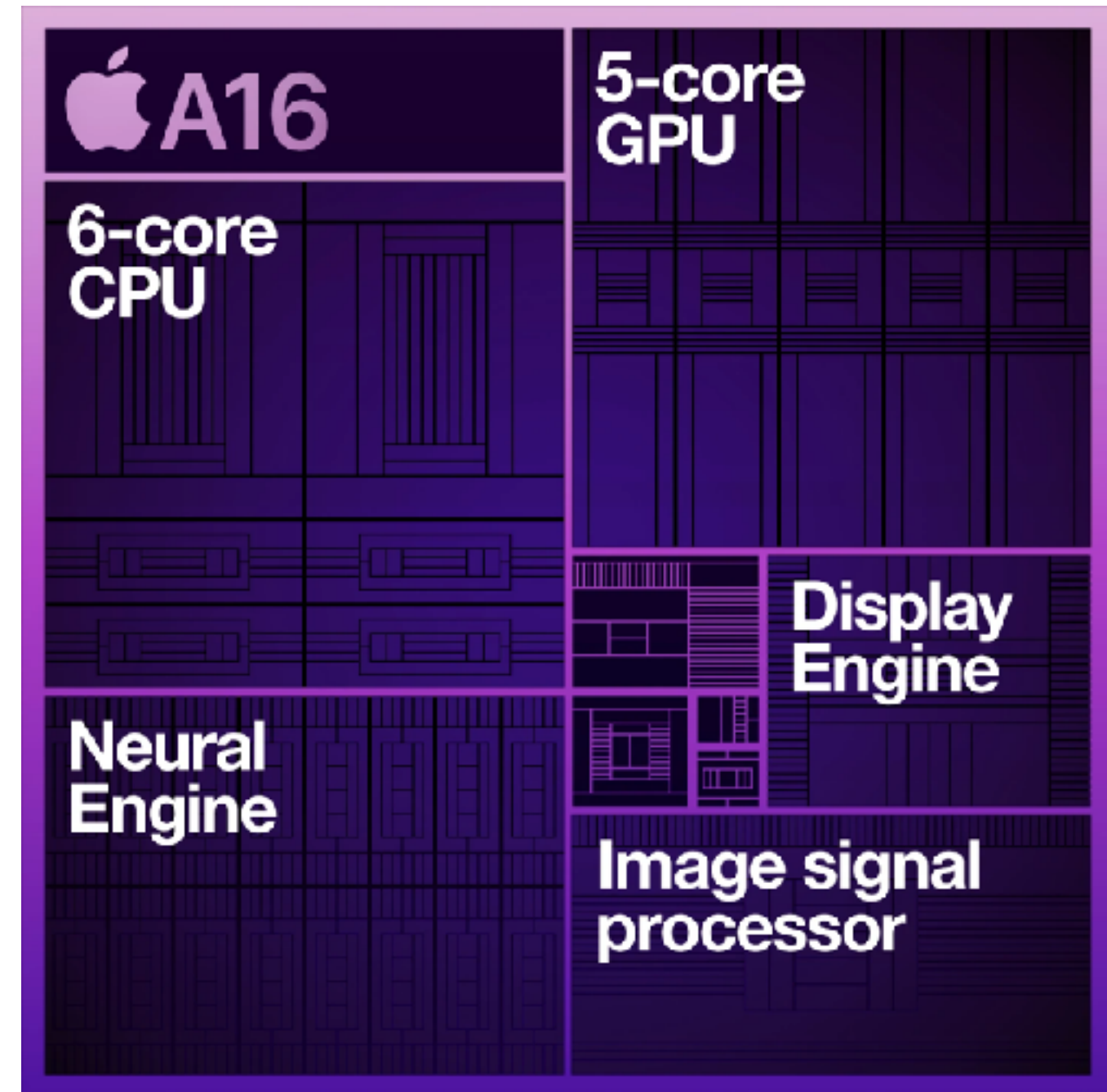


Figure 1: High-Level Architecture Diagram

Each chip is now also a collection of accelerators



**Is SoC (i.e., a processor with GPUs/
accelerators in the same package) a good
idea compared with standalone accelerators?**

Single-chip or standalone ones?

Is single-chip, heterogeneous processor a good idea?

- Potentially reduce the system interconnect bandwidth demand (still need to use intra-chip interconnect)
- Reduce the total system size and cost
- The design of the memory controller is complicated. For example, CPUs are more latency sensitive and want pairs of scalar values. GPUs/accelerators are more bandwidth demanding and want pairs of vectors
 - J. Power, M. D. Hill and D. A. Wood, "Supporting x86-64 address translation for 100s of GPU lanes," 2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA), 2014, pp. 568-578
- The power consumption per-chip is still limiting the overall performance
 - You can have only moderate processor cores with moderate GPU cores
 - You can have performance processor cores with wimpy graphic cores
 - You have to dynamically switch each part on/off
- Or you have to increase power consumption — NVIDIA's H100 goes up to 700W!!!

Is system-wide heterogeneous processing a better idea?

- Easier for heat dissipation
 - Each processor can be more powerful
 - gamers/datacenters still prefers discrete GPUs
 - Cloud TPUs are standalone ones
- System size is larger, cost of ownership can be higher
- Data movement going through system interconnects

**How do hardware accelerators change our
programming paradigms?
What're the benefits and limitations of the new
programming paradigm?**

Programming interface — NV's WMMA

```
__global__ void WMMAF16TensorCore(half *A, half *B, float *C, float *D) {

    int ix = (blockIdx.x * blockDim.x + threadIdx.x)/WARP_SIZE;
    int iy = (blockIdx.y * blockDim.y + threadIdx.y);

    wmma::fragment<wmma::matrix_a, M, N, K, half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, M, N, K, half, wmma::col_major> b_frag;
    wmma::fragment<wmma::accumulator, M, N, K, float> ab_frag;
    wmma::fragment<wmma::accumulator, M, N, K, float> c_frag;

    wmma::fill_fragment(ab_frag, 0.0f);

    // AB = A*B

    int a_col, a_row, b_col, b_row, c_col, c_row;

    a_row = ix * M;
    b_row = iy * N;

    for (int k=0; k<K_TOTAL; k+=K) {
        a_col = b_col = k;

        if (a_row < M_TOTAL && a_col < K_TOTAL && b_row < K_TOTAL && b_col <
N_TOTAL) {

            // Load the inputs
            wmma::load_matrix_sync(a_frag, A + a_col + a_row * M_TOTAL,
M_TOTAL);
```

```
                wmma::load_matrix_sync(b_frag, B + b_col + b_col * K_TOTAL,
K_TOTAL);

                // Perform the matrix multiplication
                wmma::mma_sync(ab_frag, a_frag, b_frag, ab_frag);
            }
        }

        // D = AB + C
        c_col = b_row;
        c_row = a_row;

        if (c_row < M_TOTAL && c_col < N_TOTAL) {
            wmma::load_matrix_sync(c_frag, C + c_col + c_row * N_TOTAL, N_TOTAL,
wmma::mem_row_major);

            for (int i = 0; i < c_frag.num_elements; i++) {
                c_frag.x[i] = ab_frag.x[i] + c_frag.x[i];
            }
            // Store the output
            wmma::store_matrix_sync(D + c_col + c_row * N_TOTAL, c_frag,
N_TOTAL, wmma::mem_row_major);
        }
    }
}
```

intel GNA

```
// Open selected device
status = Gna2DeviceOpen(deviceIndex);
CleanupOnError(status, deviceIndex, nullptr, GNA2_DISABLED, GNA2_DISABLED,
              "main", "Gna2DeviceOpen()");

/* Calculate model memory parameters for GnaAlloc. */
int buf_size_weights = Gna2RoundUpTo64(sizeof(weights));
// note that buffer alignment to 64-bytes is required by GNA HW
int buf_size_inputs = Gna2RoundUpTo64(sizeof(inputs));
int buf_size_biases = Gna2RoundUpTo64(sizeof(biases));
int buf_size_outputs = Gna2RoundUpTo64(H * B * 4); // (4 out vectors, H elems
in each one, 4-byte elems)

auto rw_buffer_size = Gna2RoundUp(buf_size_inputs + buf_size_outputs, 0x1000);
auto bytes_requested = rw_buffer_size + buf_size_weights + buf_size_biases;

// Allocate GNA memory (obtains pinned memory shared with the device)
uint32_t bytes_granted;
void* memory = nullptr;
status = Gna2MemoryAlloc(bytes_requested, &bytes_granted, &memory);

/* Prepare model memory layout. */
auto model_memory = reinterpret_cast<uint8_t*>(memory);

auto rw_buffers = model_memory;

auto pinned_inputs = reinterpret_cast<int16_t*>(rw_buffers);
memcpy_s(pinned_inputs, buf_size_inputs, inputs, sizeof(inputs)); // puts the
inputs into the pinned memory
rw_buffers += buf_size_inputs; // fast-forwards current pinned memory pointer
to the next free block

auto pinned_outputs = reinterpret_cast<int32_t*>(rw_buffers);
rw_buffers += buf_size_outputs; // fast-forwards the current pinned memory
pointer by the space needed for outputs

model_memory += rw_buffer_size;
auto weights_buffer = reinterpret_cast<int16_t*>(model_memory);
memcpy_s(weights_buffer, buf_size_weights, weights, sizeof(weights)); // puts
the weights into the pinned memory
model_memory += buf_size_weights; // fast-forwards current pinned memory
pointer to the next free block

auto biases_buffer = reinterpret_cast<int32_t*>(model_memory);
```

```
memcpy_s(biases_buffer, buf_size_biases, biases, sizeof(biases)); // puts the
biases into the pinned memory
model_memory += buf_size_biases; // fast-forwards current pinned memory pointer
to the next free block

/* Prepare neural network topology,
 * Single FullyConnectedAffine layer in this example. */

/* Prepare and initialize FullyConnectedAffine operation operands with GNA API
helpers */
auto inputTensor = Gna2TensorInit2D(W, B, Gna2DataTypeInt16, pinned_inputs);
auto outputTensor = Gna2TensorInit2D(H, B, Gna2DataTypeInt32, pinned_outputs);
auto weightTensor = Gna2TensorInit2D(H, W, Gna2DataTypeInt16, weights_buffer);
auto biasTensor = Gna2TensorInit1D(H, Gna2DataTypeInt32, biases_buffer);
auto activationTensor = Gna2TensorInitDisabled();

/* Create single FullyConnectedAffine operation (layer) */
auto operation = Gna2Operation{};
status = Gna2OperationInitFullyConnectedAffine(&operation, customAlloc,
                                              &inputTensor, &outputTensor,
                                              &weightTensor, &biasTensor,
                                              &activationTensor);

/* Create data-flow model with single operation (layer) */
Gna2Model model = {1, &operation};
uint32_t modelId = GNA2_DISABLED;
status = Gna2ModelCreate(deviceIndex, &model, &modelId);

// Create request configuration used for queueing inference requests
uint32_t configId = GNA2_DISABLED;
status = Gna2RequestConfigCreate(modelId, &configId);

// Set model input data buffer, operation 0, operand 0, for this sample
status = Gna2RequestConfigSetOperandBuffer(configId, 0, 0, pinned_inputs);

// Set model output data buffer, operation 0, operand 1, for this sample
status = Gna2RequestConfigSetOperandBuffer(configId, 0, 1, pinned_outputs);

// [optional] Set acceleration mode automatic (software emulation used if no
hardware detected)
status = Gna2RequestConfigSetAccelerationMode(configId,
Gna2AccelerationModeAuto);

// Enqueue inference request (non-blocking call)
```

Lessons learned from accelerators?

- Accelerators “lift up” the roofline
 - Applications/compute kernels with higher arithmetic densities may be feasible
 - NN is feasible after GPGPU
 - Trade “complexity” with parallelism
 - Applications are more likely to be memory-bound
 - Your software should try to avoid frequent memory access
 - Try to use memory closer to the processing elements
 - The hardware design must not ignore the importance of memory bandwidth
- The most “efficient” system design must land on the “turning point” of your roofline model
 - TPU’s 167GB/sec memory bandwidth is an example

Paper readings: Why, what, and how

- Why — the most important thing of a research paper
 - The big problem/question this paper is trying to address/answer
 - Why should you, as a reader, care about this paper
- What — novel ideas or interesting things to the why
 - Novel solutions to the why
 - Novel high-level design of the system
 - Novel use of an existing mechanism to solve a new problem
- How — implementation of the paper (not a contribution, but make the paper more convincing)
 - How does the implementation reflects the what?

Neural Acceleration for General-Purpose Approximate Programs

Hadi Esmaeilzadeh, Adrian Sampson, Luis Ceze, Doug Burger*

University of Washington and Microsoft*

In 45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2012)

What's the why of "Neural Acceleration for General-Purpose Approximate Programs"

Why "Neural Acceleration for General-Purpose Approximate Programs"

- ASICs/accelerators are energy-efficient, but hard-to-program
- FPGAs are slow if memory accesses are frequent
- GPUs do not work well if the workload is not regular
- Many applications tolerate inexact computation

1. Introduction

Energy efficiency is a primary concern in computer systems. The cessation of Dennard scaling has limited recent improvements in transistor speed and energy efficiency, resulting in slowed general-purpose processor improvements. Consequently, architectural innovation has become crucial to achieve performance and efficiency gains [10].

However, there is a well-known tension between efficiency and programmability. Recent work has quantified three orders of magnitude of difference in efficiency between general-purpose processors and ASICs [21, 36]. Since designing ASICs for the massive base of quickly changing, general-purpose applications is currently infeasible, practitioners are increasingly turning to programmable accelerators such as GPUs and FPGAs. Programmable accelerators provide an intermediate point between the efficiency of ASICs and the generality of conventional processors, gaining significant efficiency for restricted domains of applications.

Programmable accelerators exploit some characteristic of an application domain to achieve efficiency gains at the cost of generality. For instance, FPGAs exploit copious, fine-grained, and irregular parallelism but perform poorly when complex and frequent accesses to memory are required. GPUs exploit many threads and SIMD-

style parallelism but lose efficiency when threads diverge. Emerging accelerators, such as BERET [19], Conservation Cores and Qs-

Tolerance to approximation is one such program characteristic that is growing increasingly important. Many modern applications—such as image rendering, signal processing, augmented reality, data mining, robotics, and speech recognition—can tolerate inexact computation in substantial portions of their execution [7, 14, 28, 41]. This tolerance can be leveraged for substantial performance and energy gains.

What this paper proposed?

- Replacing the original algorithm with a machine learning model
- Learning algorithm, language/compilation framework, architectural interface

This paper introduces a new class of programmable accelerators that exploit approximation for better performance and energy efficiency. The key idea is to *learn* how an original region of approximable code behaves and replace the original code with an efficient computation of the learned model. This approach contrasts with previous work on approximate computation that extends conventional microarchitectures to support selective approximate execution, incurring instruction bookkeeping overheads [1, 8, 11, 29], or requires vastly different programming paradigms [4, 24, 26, 32]. Like emerging flexible accelerators [18, 19, 47, 48], our technique automatically offloads code segments from programs written in mainstream languages; but unlike prior work, it leverages changes in the semantics of the offloaded code.

We have identified three challenges that must be solved to realize effective trainable accelerators:

1. A **learning algorithm** is required that can accurately and efficiently mimic imperative code. We find that neural networks can approximate various regions of imperative code and propose the Parrot transformation, which exploits this finding (Section 2).
2. A **language and compilation framework** should be developed to transform regions of imperative code to neural network evaluations. To this end, we define a programming model and implement a compilation workflow to realize the Parrot transformation (Sections 3 and 4). The Parrot transformation starts from regions of approximable imperative code identified by the programmer, collects training data, explores the topology space of neural networks, trains them to mimic the regions, and finally replaces the original regions of code with trained neural networks.
3. An **architectural interface** is necessary to call a neural processing unit (NPU) in place of the original code regions. The NPU we designed is tightly integrated with a speculative out-of-order core. The low-overhead interface enables acceleration even when fine-grained regions of code are transformed. The core communicates both the neural configurations and run-time invocations to the NPU through extensions to the ISA (Sections 5 and 6).

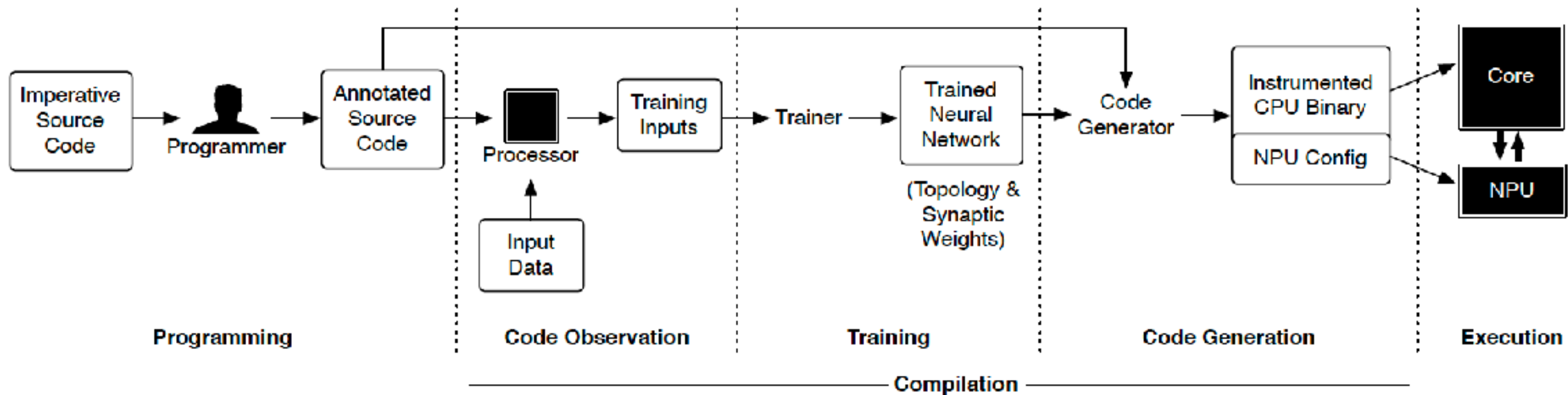


Figure 1: The Parrot transformation at a glance: from annotated code to accelerated execution on an NPU-augmented core.

The overall architecture

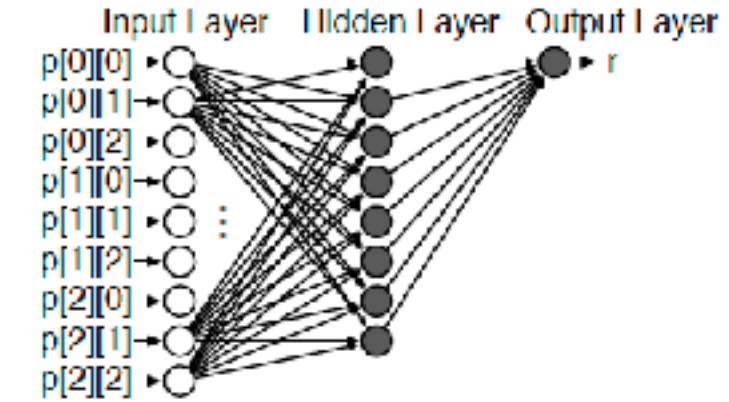
```

1  float sobel [[PARROT]] (float[3][3] p){
    float x, y, r;
3   x = (p[0][0] + 2 * p[0][1] + p[0][2]);
    x = (p[2][0] + 2 * p[2][1] + p[2][2]);
5   y = (p[0][2] + 2 * p[1][2] + p[2][2]);
    y = (p[0][0] + 2 * p[1][1] + p[2][0]);
7   r = sqrt(x * x + y * y);
    if (r >= 0.7071) r = 0.7070;
9   return r;
}

void edgeDetection(Image& srcImg, Image& dstImg){
2   float[3][3] p; float pixel;
    for(int y = 0; y < srcImg.height; ++y)
4       for(int x = 0; x < srcImg.width; ++x)
            srcImg.toGrayscale(x, y);
6       for(int y = 0; y < srcImg.height; ++y)
            for(int x = 0; x < srcImg.width; ++x){
8           p = srcImg.build3x3Window(x, y);
            pixel = sobel(p);
10          dstImg.setPixel(x, y, pixel);
        }
12 }

```

(a) Original implementation of the Sobel filter



(b) The sobel function transformed to a $9 \rightarrow 8 \rightarrow 1$ NN

```

void edgeDetection(Image& srcImg, Image& dstImg){
2   float[3][3] p; float pixel;
    for(int y = 0; y < srcImg.height; ++y)
4       for(int x = 0; x < srcImg.width; ++x)
            srcImg.toGrayscale(x, y);
6       for(int y = 0; y < srcImg.height; ++y)
            for(int x = 0; x < srcImg.width; ++x){
8           p = srcImg.build3x3Window(x, y);
            NPU_SEND(p[0][0]); NPU_SEND(p[0][1]); NPU_SEND(p[0][2]);
10          NPU_SEND(p[1][0]); NPU_SEND(p[1][1]); NPU_SEND(p[1][2]);
            NPU_SEND(p[2][0]); NPU_SEND(p[2][1]); NPU_SEND(p[2][2]);
12          NPU_RECEIVE(pixel);
            dstImg.setPixel(x, y, pixel);
14      }
}

```

(c) parrot transformed code; an NPU invocation replaces the function call

Figure 2: Three stages in the transformation of an edge detection algorithm using the Sobel filter.

Let's try a simple problem — binary MM

0	1	0	1
0	1	1	0
1	0	1	1
0	0	0	1

x

1	0
0	0
1	1
1	1

=

1	1
1	1
3	2
1	1

→

0	0
0	0
1	1
0	0

How're you going to implement the idea of "NPU" in existing systems?

How

- Something hard to find in introduction, but we don't care that much unless we want to implement/reproduce that
- They simulated the system

7.1. Benchmarks and the Parrot Transformation

Table 1 lists the benchmarks used in this evaluation. These benchmarks are all written in C. The application domains—signal processing, robotics, gaming, compression, machine learning, and image processing—are selected for their usefulness to general applications and tolerance to imprecision. The domains are commensurate with evaluations of previous work on approximate computing [1, 11, 28, 29, 41, 43].

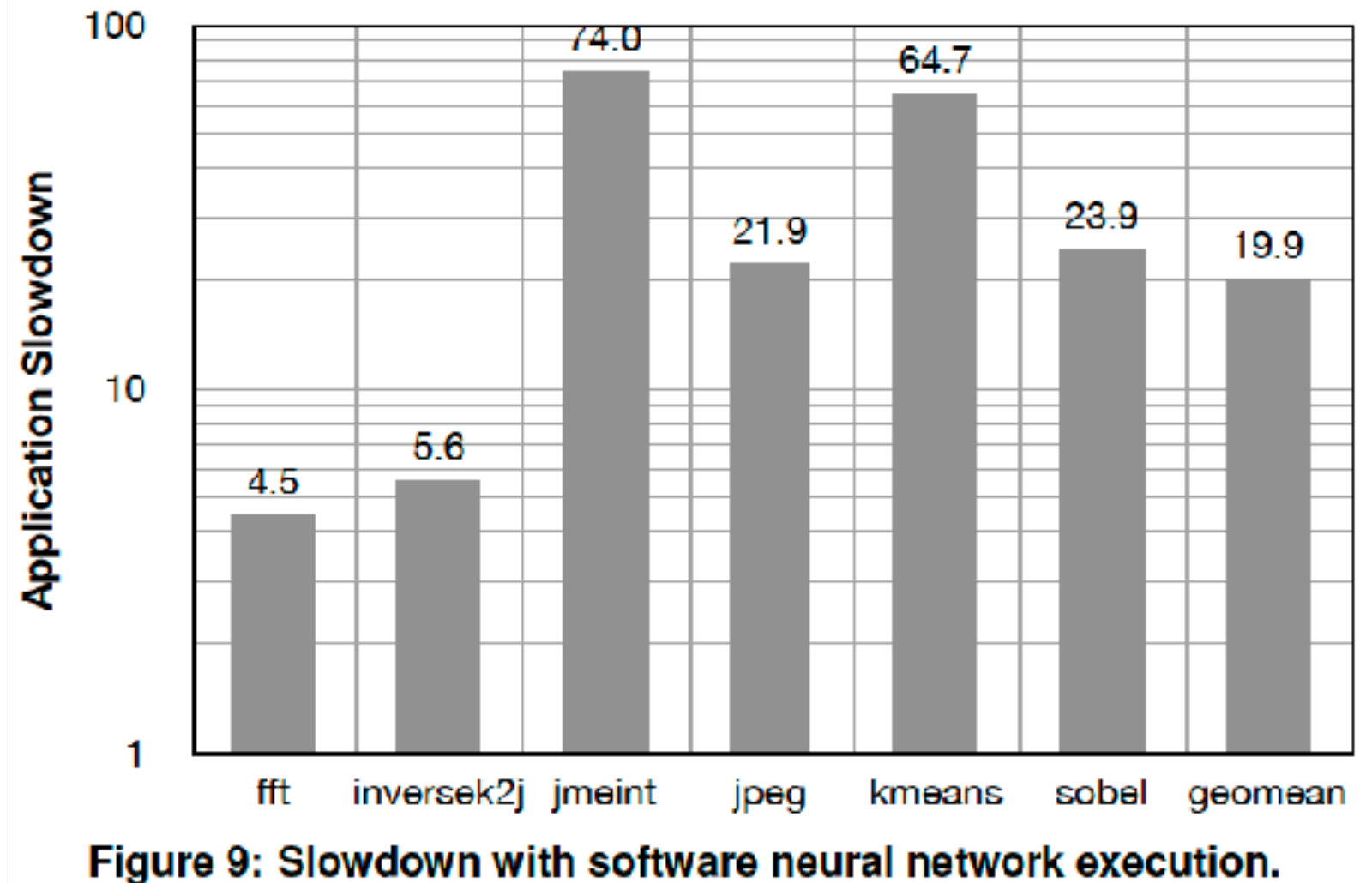
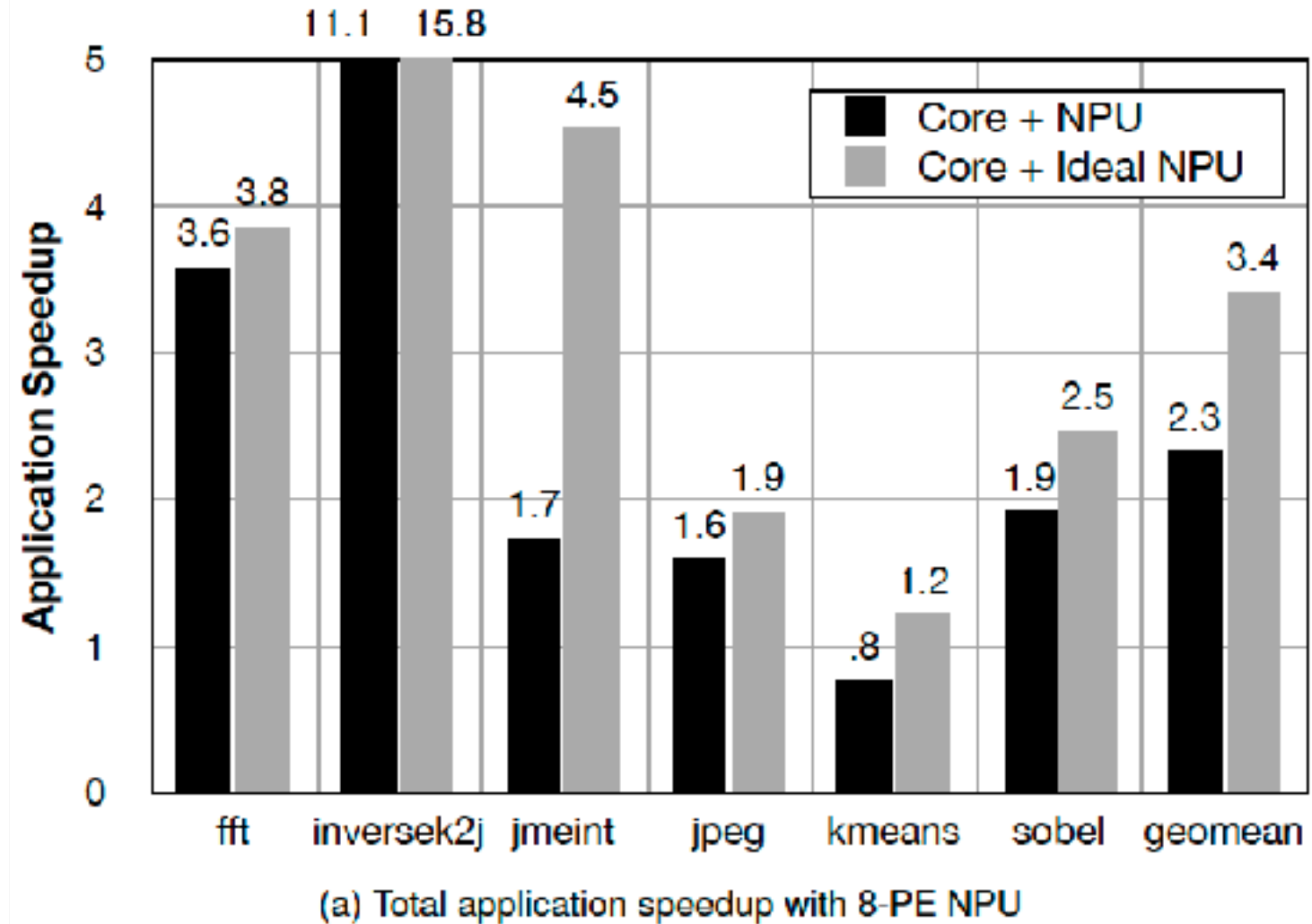
Table 1 also lists the input sets used for performance, energy, and accuracy assessment. These input sets are different from the ones used during the training phase of the Parrot transformation. For applications with random inputs we use a different random input set. For applications with image input, we use a different image.

Code annotation. The C source code for each benchmark was annotated as described in Section 3: we identified a single pure function with fixed-size inputs and outputs. No algorithmic changes were made to the benchmarks to accommodate the Parrot transformation. There are many choices for the selection of target code and, for some programs, multiple NPUs may even have been beneficial. For the purposes of this evaluation, however, we selected a single target region per benchmark that was easy to identify, frequently executed as to allow for efficiency gains, and amenable to learning by a neural network. Qualitatively, we found it straightforward to identify a reasonable candidate function in each benchmark.

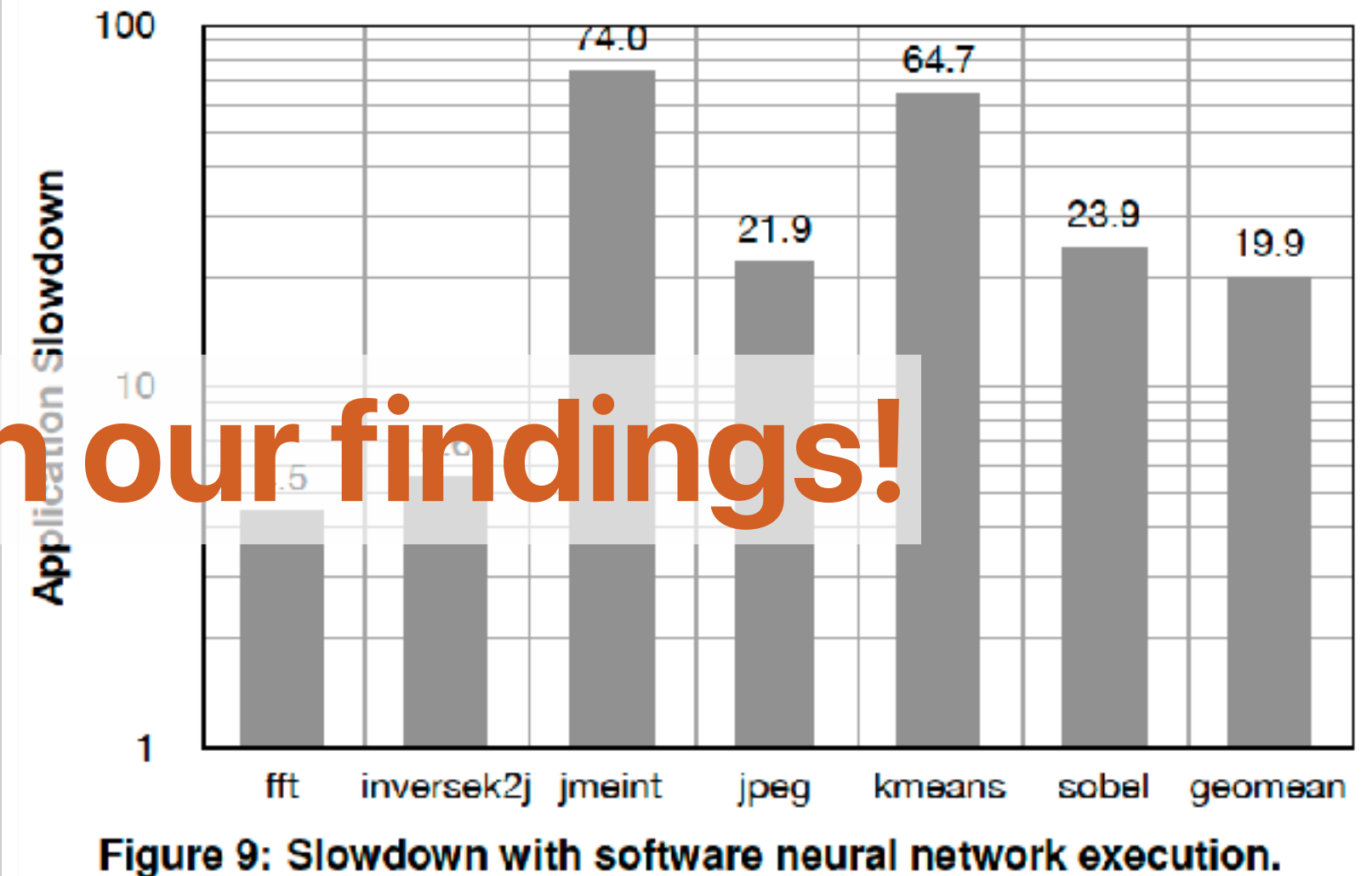
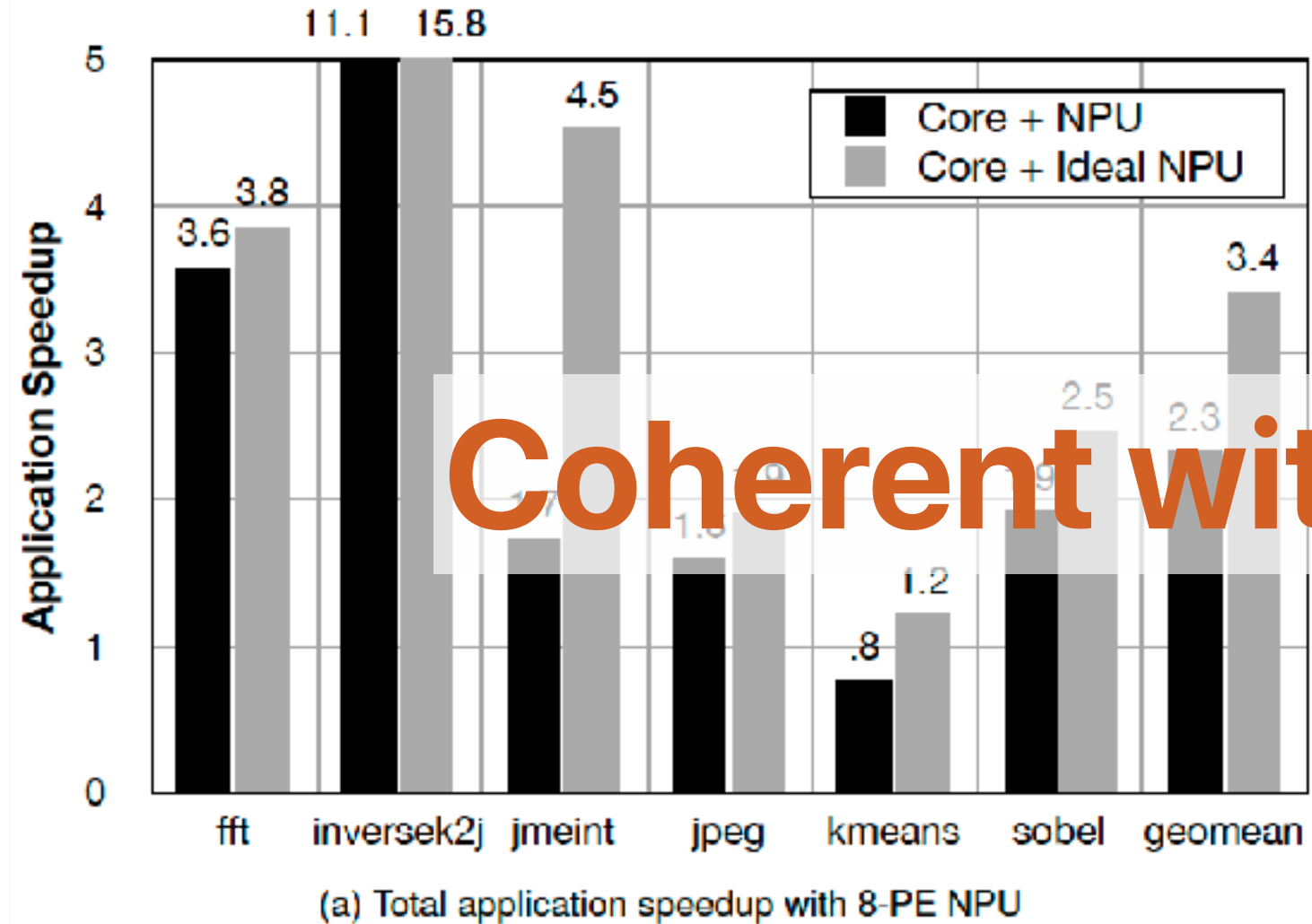
7.2. Experimental Setup

Cycle-accurate simulation. We use the MARSSx86 cycle-accurate x86-64 simulator [35] to evaluate the performance effect of the Parrot transformation and NPU acceleration. Table 2 summarizes the microarchitectural parameters for the core, memory subsystem, and NPU. We configure the simulator to resemble Intel's Penryn microarchitecture, which is an aggressive out-of-order design. We augment MARSSx86 with a cycle-accurate NPU simulator and add support for NPU queue instructions through unused x86 opcodes. We use C assembly inlining to add the NPU invocation code. We compile the benchmarks using GCC version 4.4.6 with the `-O3` flag to enable aggressive compiler optimizations. The baseline in all of the reported results is the execution of the entire benchmark on the core without the Parrot transformation.

The idea works more efficiently if accelerator exists



The idea works more efficiently if accelerator exists



Coherent with our findings!

Roogle Project Meetings



- Make an appointment through the Google Calendar
- Available resources
 - SmartSSD, Edge TPUs, CUDA GPUs, Tensor Cores, or something else
- Project ideas
 - Accelerating applications through AI/ML accelerators
 - Accelerating applications through intelligent storage devices
 - Accelerating applications through innovative parallel programming models that hardware accelerators enable
 - Anything related to what we discussed in this class!

Electrical Computer Science Engineering

277

つくづく

